

ИИ / РАЗРАБОТКА ИСКУССТВЕННОГО ИНТЕЛЛЕКТА / СРЕДСТВА РАЗРАБОТКИ / КОДИРОВАНИЕ
ВИБРАЦИИ

“Код следует создавать заново, а не поддерживать”: Codeplain обосновывает необходимость разработки на основе спецификаций

Феникс из греческого мифа сгорает и возрождается из собственного пепла - метафора, которую этот основатель применяет к коду, сгенерированному искусственным интеллектом.

25 июня 2026 г. в 13:47 от [Пол Соэрс](#)



Летний флаг для Unsplash+

ИИ генерирует код быстрее, чем команды успевают его проверять. По
нию небольшого, но растущего числа разработчиков, решение

Душан Омерчевич — генеральный директор и соучредитель Codeplain, компании, которая делает ставку на спецификационно-ориентированную разработку как основу для создания и поддержки программного обеспечения эпохи искусственного интеллекта. Основанная в Любляне, Словения, в начале 2025 года, Codeplain тихо вышла на рынок в сентябре того же года, пообещав «спецификационно-ориентированную генерацию готового к использованию кода.»

В основе этого подхода лежит Plain, язык спецификаций с открытым исходным кодом, который использует структурированные, удобочитаемые документы в качестве единственного достоверного источника информации о том, как должно быть устроено программное обеспечение и как оно должно работать. Идея заключается в том, что если что-то не работает или требует изменения, вы редактируете спецификацию, а не код, и Codeplain генерирует реализацию с нуля.

specification in <code>***plain</code>	<code>*codeplain generated python code</code>
<pre>trello-client.plain --- description: 'Trello client application' import: - python-console-app-template - saas-client-reqs --- ***definitions*** - :API: is the API of the 3rd party SaaS service. - :ListOfUsers: is the list of users of the 3rd party SaaS service. ***implementation reqs*** - The resource trello-openapi.json describes :API:. ***test reqs*** - The resource trello-credentials.json contains credentials that can be used for testing. ***functional specs*** - :App: should take credentials json as the only positional argument. - :App: should authenticate with :API:. - :App: should fetch :ListOfUsers: from :API: and print them to the console.</pre>	<pre>trello-client.py def main(args=None): try: logger = setup_logging() logger.info("Ap</pre>

Спецификация Plain для клиентского приложения Trello (слева) и код Python, сгенерированный Codeplain на ее основе (справа).

Теперь Омерчевич продвигает эту идею дальше. В интервью *The New Stack* он утверждает, что по мере того, как ИИ генерирует все большие объемы кода, основная сложность смещается с написания программного обеспечения на проверку и поддержку, а проверка спецификаций, которые отражают

«Наш тезис заключается в том, что код не нужно поддерживать — его нужно генерировать заново. Спецификации должны проверяться, и именно спецификации вы поддерживаете в актуальном состоянии».

«Наша идея заключается в том, что код не нужно поддерживать — его нужно генерировать заново, — объясняет Омерчевич. — Спецификации должны проверяться, и именно спецификации вы поддерживаете в актуальном состоянии».

To advance that mission, the company is rolling out a new open-source agentic skills framework dubbed **plain-forge**, which lets coding agents such as Claude Code, Codex, and OpenCode draft and maintain Plain specs through conversation — effectively automating the part of spec-driven development that previously required the most human effort.

Alongside the launch, Codeplain also announced that it has raised \$3M in funding to date from backers including **GapMinder VC** and **Silicon Gardens**.

TRENDING STORIES

1. "Code should be regenerated, not maintained": Codeplain makes the case for spec-driven development
2. Your engineering org needs an AI slop registry
3. Loops are replacing prompts. Verification is about to be your biggest problem.
4. Tutorial: Building a Real-time Flight Tracker with Gemini Function Calling Capability
5. Small Language Models Are Driving a New Wave of Edge Native AI

a joy of specs: a new development paradigm

structured specifications generated from natural-language prompts. GitHub, meanwhile, **followed with Spec Kit**, an open-source toolkit designed to make specs executable artifacts that AI agents can act on directly.

The underlying concept, however, predates both. **SpecLang** was a GitHub Next research project that began exploring in 2023 whether plain English could serve as a true programming medium — one where AI handles the translation from human intent to working code.

That lineage connects directly to Codeplain, too. **Johan Rosenkilde**, one of SpecLang's creators and a founding member of the GitHub Copilot team, **sits on Codeplain's board** and was among its earliest investors.

But despite the momentum behind spec-driven development, it seems developers don't particularly want to write specifications. Through user testing, Omerčević and his team found that while developers resist writing specs, they are generally happy to read them. A well-structured spec, it turns out, is easier to review and reason about than the code it eventually produces.

Plain-forgo is Codeplain's answer to that tension. Rather than asking developers to author specifications from scratch, it puts the coding agent to work — researching the problem, drafting the spec incrementally, and validating it against Codeplain's renderer before a human ever sees it. Crucially, it doesn't front-load the process by generating a sprawling specification upfront — an approach Omerčević is openly critical of.

"We don't throw 200 lines of specs at a developer; we do it iteratively," he says.

Each small spec generates working software that the developer can immediately check and respond to, building familiarity with the spec one feature at a time rather than confronting a finished document they had no hand in shaping.

"By incrementally adding to the specs, the developer is building relations with the spec."

"In this way, by incrementally adding to the specs, the developer is building relations with the spec," Omerčević says. "The specs are the source of truth, and specs are the feedback to the developer that the AI understood them."

disposable output rather than a durable asset, and that the spec — not the generated code — is what teams should preserve and maintain. Omerčević says he arrived at this conclusion through his own experience building Codeplain, but found it difficult to articulate to developers whose entire professional identity is built around code.

This is where **Chad Fowler's *Phoenix Architecture*** enters the fray. Fowler, a veteran engineer and General Partner at BlueYard Capital, has spent the past six months developing a broader philosophical framework for this approach.

His inaugural post in the series back in December, titled ***Regenerative Software***, argues that AI has made code abundant and cheap, inverting decades of assumptions about what in a software system is worth preserving — and that teams clinging to existing implementations are generating technical debt faster than they realize.

It's a framework that Omerčević says gave him a vocabulary he'd been struggling to find.

"Spec-driven development was really hard to communicate to regular developers."

"Spec-driven development was really hard to communicate to regular developers," Omerčević says. "What Chad did really well is ground the story in code, saying that the only difference is that this code is no longer permanent — it's ephemeral, and it can always be regenerated from other artifacts."

The **phoenix**, for the uninitiated, is a creature of Greek myth — a bird that cyclically burns itself and is reborn from the ashes. Fowler's argument is that software systems should be designed to do the same.

In his March post, **"The Conversation Is the Commit,"** Fowler argues that when developers manually edit AI-generated code, they sever something important: the record of why that code exists and what decisions shaped it. The output changes, the reasoning behind the change is lost.

careers without a name for it.

“If we don’t move toward finally capturing this rich and important information, we are irresponsible in the face of this wave of innovation in how we build software.”

“It’s just that it was impossible, technologically, to capture provenance without extreme bureaucracy, so as an industry we all decided it wasn’t a trade worth trying to make in almost all cases,” Fowler explains to *The New Stack*. “Now the tooling opens these sorts of things up without heavy process. My argument is that if we don’t move toward finally capturing this rich and important information, we are irresponsible in the face of this wave of innovation in how we build software.”

Put another way, the specs and the reasoning behind them are now the thing worth preserving — not the code they produce.

It’s difficult to overstate how significant a cultural shift this would entail — developers, after all, have built their identities around code. So how do you convince an engineer that deletion and regeneration represent progress?

For Fowler, there are two ways, each reflecting a different kind of persuasion.

“The first is to wait, because they will see that, over time, the old way of doing things has become obsolete and they need to evolve or die,” he says. “The second, and more positive way is to show them that deletion and regeneration define, and are supported by, a new level of rigor that only this new tooling makes possible.”

Codeplain, in Fowler’s view, represents one credible attempt to build that rigor into a working product — though he’s careful to say it’s just one piece of a larger puzzle still being assembled.

“My hope is that many companies and open source developers [build it],” he says. “This is because I think this missing layer can look very different. There’s a lot of room for imagination and innovation here. I don’t want to go with any one person or

Up to spec



Codeplain founders Predrag Radenkovic (CTO) and Dusan Omercevic (CEO)

Omerčević is no stranger to building and shipping enterprise software — he previously **founded Cleanshelf**, a SaaS management platform **he sold to LeanIX in 2021**, which in turn was subsequently **acquired by SAP**. Codeplain, then, is his next act — and it already has customers putting it to work.

Incode, an identity verification service provider, uses Codeplain to build and maintain integrations with external data providers — the kind of work that involves constant API research, rapidly changing external systems, and a high tolerance for things breaking unexpectedly.

It's that last problem — things breaking — that Omerčević is most animated about. Because specs encode intent rather than implementation detail, when an external API changes and breaks an integration, Codeplain can often fix it by regenerating the code from the exact same spec, unchanged. The spec didn't break; only the code did.

but specs don't.

There are also hard economic arguments behind the approach. Omerčević says that coding agents that generate specs use five to ten times fewer tokens than when generating code directly — and that because specs are cognitively less demanding for the agent, it can handle larger, more complex problems within the same context window. For the code generation step itself, Codeplain uses faster, cheaper models such as Gemini Flash rather than frontier models, keeping costs down.

The analogy Omerčević reaches for is the TypeScript compiler: Claude could generate JavaScript directly from TypeScript, but why would you when a specialized tool does it faster and more cheaply?

“Let the specialized tools do what they are really good at,” he says. “And let Claude do what it is really good at — and that is research.”

TNS



Paul is an experienced technology journalist covering some of the biggest stories from Europe and beyond, most recently at TechCrunch where he covered startups, enterprise, Big Tech, infrastructure, open source, AI, regulation, and more. Based in London, these days Paul...

[Read more from Paul Sawers →](#)

TNS owner Insight Partners is an investor in: LeanIX.

SHARE THIS STORY



TRENDING STORIES

1. "Code should be regenerated, not maintained": Codeplain makes the case for spec-driven development
2. Your engineering org needs an AI slop registry

.oops are replacing prompts. Verification is about to be your biggest problem.

5. Small Language Models Are Driving a New Wave of Edge Native AI

INSIGHTS FROM OUR SPONSORS



Deploying Applications to Kubernetes from Source Code: Five Practical Approaches Compared

22 June 2026

Why Crossplane Alone Is Not Database Management, and What Platform Teams Need Instead

19 June 2026

Multi-Account AWS Is a Control-Plane Problem — And Most Teams Learn That the Hard Way

13 May 2026



A 15-Second Health Check for Your Heroku Connect Data Pipeline

24 June 2026

Heroku March 2026 Update

19 March 2026

Modernizing the Command Line: Heroku CLI v11

19 March 2026



TiDB Completes Independent Security Assessment by NCC Group

June 2026

The Fintech AI Platform: Why Agents Need One Database, Not Five

23 June 2026

TNS DAILY NEWSLETTER

Receive a free roundup of the most recent TNS articles in your inbox each day.

svo1975pvn1982@gmail.com

SUBSCRIBE

The New Stack does not sell your information or share it with unaffiliated third parties. By continuing, you agree to our [Terms of Use](#) and [Privacy Policy](#).

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data

Open Source

Networking

Storage

Security

Software Development

WebAssembly

OPERATIONS

AI Operations

CI/CD

Cloud Services

DevOps

Kubernetes

Observability

Operations

Platform Engineering

CHANNELS

Podcasts

Ebooks

Events

Webinars

Newsletter

TNS RSS Feeds

THE NEW STACK

About / Contact

Sponsors

Advertise With Us

Contributions

 roadmap.sh

Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

Frontend Developer Roadmap

Backend Developer Roadmap

Devops Roadmap

FOLLOW TNS



© The New Stack 2026

Disclosures

Terms of Use

Advertising Terms & Conditions

Privacy Policy

kie Policy

